

School of Computer Science and Engineering, UNSW



## WK02-01: Blockchain Platforms

Software Architecture for Blockchain Applications,  
COMP6452

Never Stand Still

Salil Kanhere, Nadeem Ahmed

# Agenda

## Popular Blockchain Platforms

- Bitcoin
- Ethereum
- Hyperledger

- We'll discuss 3 popular BC platforms. Under each platform we'll revisit TXs, blocks, and ledgers structures in more details.
- Bitcoin and Ethereum are public BCs, while Hyperledger Fabric is a private (aka consortium) BC.
- This will be only an introduction. Some of the concepts like mining and consensus will be discussed in detail later.

## Cryptocurrency

Digital currency baked into a blockchain & secured by cryptography

- Accounting & validation rules are hard-coded in the platform's base layer by developers
- A platform typically has one base cryptocurrency
- aka native currency

Not centrally issued, e.g., Bitcoin & Ethereum

Can be mined or purchased from cryptocurrency exchanges

Usually only on public blockchains

Usually don't represent other rights/assets

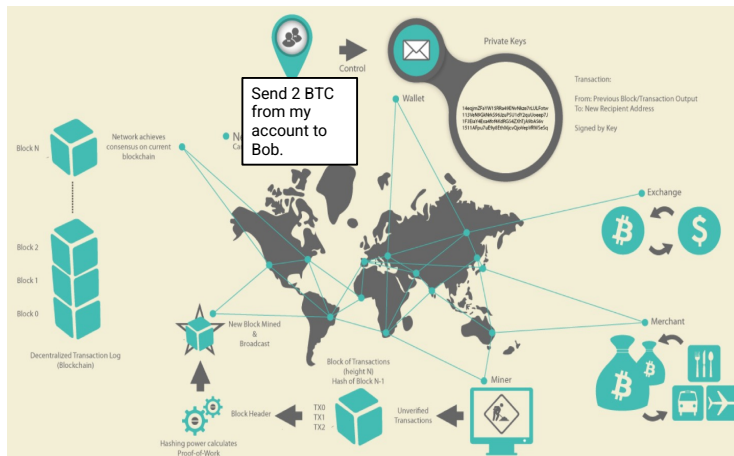
WK 02-01

3



- Here's a single slide introduction to cryptocurrencies which are digital assets (aka crypto-assets).
- A cryptocurrency is baked into a blockchain & secured by cryptography.
  - Accounting & validation rules are hard-coded in the platform's base layer (aka BC protocol) by developers.
  - A platform typically has one base cryptocurrency. However, other forms of currencies/tokens may exist for governance, staking, etc.
  - aka native currency
- Cryptocurrencies are not centrally issued by a central party like a central/reserve bank, e.g., in Bitcoin & Ethereum, new Cryptocurrency can only be minted. And any miner can do this.
- They can be mined by joining the network or purchased from cryptocurrency exchanges.
- They usually exist only on public blockchains.
- In most cases, they usually don't represent other rights/assets like a land title. Coloured coins are an exception whereas a Bitcoin UTXO represents some other asset.

## 1<sup>st</sup> Gen Blockchains - Cryptocurrency



Source: Andreas M. Antonopoulos, Mastering Bitcoin-Unlocking Digital Cryptocurrencies

### Users:

- Create TXs
- Sign them
- Announce TXs to network

### Miners:

- Receive TXs
- Include them in a new block
- (try to) append new block to the BC
- Tx finalised when block is appended and confirmed

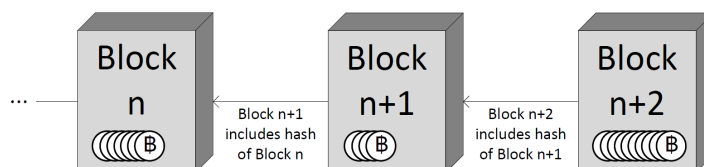
### Exchanges:

- Facilitate trading between Fiat and Cryptocurrencies

- 1<sup>st</sup> generation blockchains focused only on cryptocurrencies.
- Bitcoin is the 1<sup>st</sup> cryptocurrency on a blockchain.
- Bitcoin ecosystem consists of 3 main roles, namely users, miners, and exchanges.
- Users create TXs, sign them, & announce them to the Bitcoin network.
- Miners receive TXs, include them in a new block, and try to append a new block to the chain of blocks. If successful, the block and its TXs are considered to have taken place. You then wait for it to be confirmed.
- Exchanges are used to trade Bitcoin with other fiat and cryptocurrencies.
- The other role is to follow the protocol in performing other functions such as consensus and information propagation.



## Bitcoin - BTC



1st cryptocurrency (BTC) built on the idea of a blockchain

- 2008 white paper by Satoshi Nakamoto – Paper never used the word “blockchain”
- Implementation in Jan. 2009

Blockchain keeps track of the ownership of portions of that currency (Satoshi)

Miners compete to build the next block

Average time between blocks, called inter-block time, is ~10-min

- But variation of times is high

WK 02-01

5



- Bitcoin is the 1st cryptocurrency built on the idea of a BC. Both the BC network and currency are called Bitcoin and the symbol is BTC.
- It was proposed in a 2008 white paper by a pseudonym called Satoshi Nakamoto.
  - The white paper never used the word BC. But as Bitcoin followed the linked-list structure forming a chain of blocks, BC became a synonym for the data structure that links blocks. It is also used to refer to the network of nodes too.
- Implementation appeared in Jan. 2009.
  - On 3 January 2009, the Bitcoin network was created when Nakamoto mined the starting block of the chain, known as the *genesis block*.
  - The software was published by Satoshi Nakamoto under the name "Bitcoin", and later renamed to "Bitcoin Core" to distinguish it from the network, It's also known as the Satoshi client.
- Bitcoin BC keeps track of the ownership of portions of cryptocurrency, i.e., Bitcoins (& Satoshis) associated with each address.
- As discussed in the last lecture, a block mainly contains a bunch of TXs and a reference to the previous block.
- Miners compete to build the block and a new block appears after several minutes.
- The time between 2 blocks is called *inter-block time*.
- The average inter-block time is 10-min. However, it changes widely from a couple of minutes to even as high as 20 - 30 min. You can observe this in real time using a blockchain explorer.

# Snapshot of Bitcoin Network

Source: <https://bitnodes.io>

**REACHABLE BITCOIN NODES**  
Updated: Thu Jun 5 16:34:49 2025 AEST

**22373 NODES**

**CHARTS**

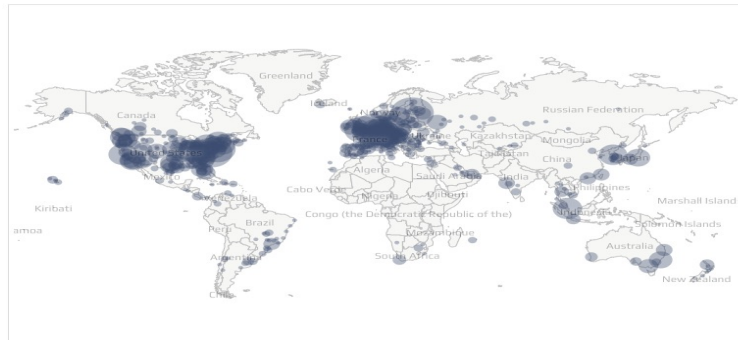
IPv4: +5.0% / IPv6: +1.4% / .onion: +1.9%

Top 10 countries with their respective number of reachable nodes are as follows.

| RANK | COUNTRY                            | NODES          |
|------|------------------------------------|----------------|
| 1    | <a href="#">n/a</a>                | 14319 (64.00%) |
| 2    | <a href="#">United States</a>      | 2325 (10.39%)  |
| 3    | <a href="#">Germany</a>            | 1285 (5.74%)   |
| 4    | <a href="#">France</a>             | 590 (2.64%)    |
| 5    | <a href="#">Finland</a>            | 394 (1.76%)    |
| 6    | <a href="#">Canada</a>             | 370 (1.65%)    |
| 7    | <a href="#">Netherlands</a>        | 361 (1.61%)    |
| 8    | <a href="#">United Kingdom</a>     | 315 (1.41%)    |
| 9    | <a href="#">Switzerland</a>        | 229 (1.02%)    |
| 10   | <a href="#">Russian Federation</a> | 177 (0.79%)    |

[All \(99\)»](#)

**NOTE** / The data above includes reachable nodes only. [View global nodes here](#) »



Map shows concentration of reachable Bitcoin nodes found in countries around the world.

**LIVE MAP**

WK 02-01

6



- This is a visualisation of the node distribution in the Bitcoin network as of June 2025.
- You can go to this website to see more recent details.
- As per the statistics collected, there are ~22K nodes in the network. This number can vary a bit and is typically above 20K.
- The map shows the concentration of nodes where location can be estimated. So, for a large portion of nodes (64%), we don't know the location.
- It also shows the nodes with IPv4 and IPv6 addresses and the ones that are using Tor
- You can get a bit more details on the live map by visiting the bitnodes.io site. Go to this website and explore a bit.
- ASNs - Autonomous System number is an identifier of the ISP

## Accounts & States

An account is associated with a cryptographic key pair

- Public key – Used to create the address of an account
- Private key – Sign TXs sent from the account

State of the blockchain

- Account balances of all users – World state
- Result from the genesis block (very 1st block) & set of TXs included since
  - Some accounts can be pre-loaded with an initial account balance at the genesis

As TXs are grouped into blocks, when a new block is added the entire system moves from one discrete state to another

| ID          | Asset                              |
|-------------|------------------------------------|
| Alice       | 500                                |
| Bob         | 1000                               |
| Charlie     | 500                                |
| Dave        | Plot 123 @ 2015                    |
| Sweet Mango | Bowen QUE,<br>Org. Cert #<br>45781 |

- Let's discuss a few definitions which can be generalised to other BCs too.
- A Bitcoin account is associated with a cryptographic key pair, where
  - The public key is used to create the address of an account.
  - The private key is used to sign TXs sent from the account (i.e., authenticate TXs).
  - An account has only a single public-private key pair.
- The state of the BC is the account balance of all users. This is also referred to as the *Global state* or *World state*.
  - Account balance is the sum of BTCs (named UTXOs, discussed in the next slides) that an account has control over. Alice, Bob, and Charlie's balances but also other state maintained by the network (keeping track of other data is more complicated, and will be discussed later).
    - Each UTXO is bound to the owner's public key
  - The state captures results from the genesis block (the very 1st block) & set of TXs included since.
    - Some accounts might be pre-loaded with an initial account balance in the Genesis block.
- As TXs are grouped into blocks, the entire system moves from one discrete state to another through the creation of a new block.

## Fun Facts

### Bitcoin Block 0

Mined on January 04, 2009 05:15:05 • All Blocks

Satoshi

Notable Block

**Coinbase Message** • EThe Times 03/Jan/2009 Chancellor on brink of second bailout for banks

#### Bitcoin Genesis

On January 3rd 2009, the Bitcoin network was created when Satoshi Nakamoto (the project's mysterious creator) mined the "Genesis" block. The 50 bitcoin coinbase reward is unredeemable, as it was omitted from the transaction database. This means any attempt to spend it would be rejected by the network. Whether this was intentional or not still remains unknown.

A total of 0.00 BTC (\$0.00) were sent in the block with the average transaction being 0.00000 BTC (\$0.00). Satoshi earned a total reward of 50.00 BTC \$0.00. The reward consisted of a base reward of 50.00 BTC \$0.00 with an additional 0.00000 BTC (\$0.00) reward paid as fees of the 1 transactions which were included in the block.

- Genesis block in Bitcoin is an example of an empty block with no other TX except the Coinbase Tx.
- Can you find out when the Block No 1 was mined on the Bitcoin platform?
- Which is the first block on Bitcoin that involved transaction fees?

## Transactions (TXs)

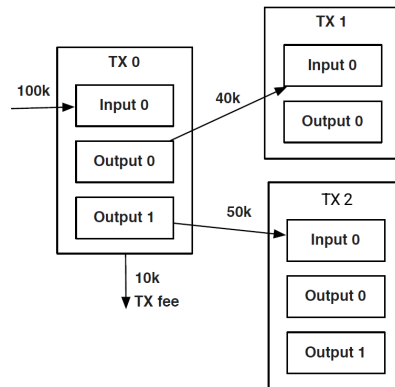
Transfer currency from source addresses to destination addresses

Contains 1+ inputs & 1+ outputs

- If sum of outputs is less than sum of inputs, the difference is a fee to the miner
- TX fee is an incentive for miners to contribute computing power & storage

Contains proof of ownership for each input, in the form of a digital signature of owner

TX output is bind to the (new) owner's public key



- Bitcoin TXs have an interesting structure and are used to transfer Bitcoins from source addresses to destination addresses, like from one bank account to another.
- A TX contains one or more inputs & one or more outputs. E.g., TX0 has 1 input and 2 outputs.
  - You can think of this as paying a merchant using multiple notes and coins and getting the balance in the same way.
- Each of those outputs can be spent later in another TX. Think of the balance you get from the merchant, which can be spent somewhere else.
- The difference between input & output values is taken as the TX fee by the miner. E.g., here 100,000 Satoshis 100K comes in. A Satoshi is like a cent. The total value of outputs is 90K. The remaining 10K is the TX fee.
  - Satoshi is the smallest unit of BTC and it's one 100 millionth of a single Bitcoin (eight decimal places as 0.00000001 BTC). Millibitcoin (mBTC) is 1/1000 of a bitcoin.
- These TX fees are the incentive for miners to contribute computing power, storage, and bandwidth.
- A TX contains proof of ownership for each input, in the form of a digital signature of the owner. That is the owner needs to sign the TX using his/her private key which the Bitcoin miners can validate using the owner's public key.
- An output arising from a TX is linked to an address to confirm its ownership. E.g., when Alice pays 2 BTC to Bob, the output of the transaction will contain Bob's address. Then to spend that 2 BTC, Bob needs to use his private key to sign the corresponding TX.

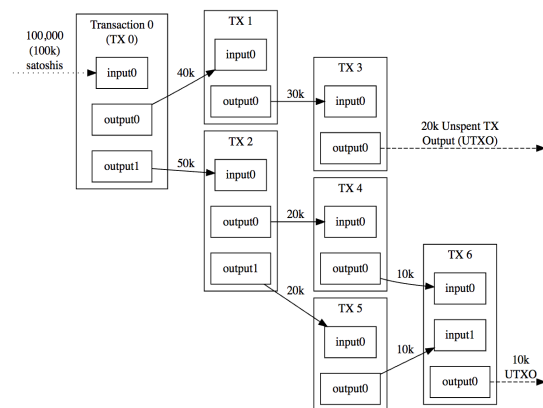
## Transaction Format

### Linked TXs

- Outputs of TXs become inputs of a new TX
- Outputs of previous transactions become unspent Transaction Outputs (UTXO) that can be spent in future transactions

### Bitcoin addresses don't contain "coins" balance

- Different to a bank account
- Similar to notes/coins in your physical wallet
- Store UTXOs



- Bitcoin TXs are linked where the output of one or more TXs becomes inputs to a new TX.
  - Different inputs and outputs of the same TX are identified based on their index number.
- However, compared to a typical account and balance maintained by a bank, Bitcoin addresses don't contain "coins".
- Instead, these unspent outputs are called UTXO (Unspent TX Output). UTXOs are more like notes and coins in your wallet.
  - 20k output from TX 3 is a UTXO.
  - Values in the figure are in Satoshis – A Satoshi is the smallest denomination of bitcoin, equivalent to 100 millionth of a Bitcoin ( $10^{-8}$ ).

## UTXO

Balance of an address/account

- Sum of values of all of UTXOs associated with the address

Once used, a UTXO is fully consumed and replaced with a new UTXO

(World) State of the blockchain

- All the UTXOs in system

A UTXO is immutable and atomic

- Once a UTXO is created, it cannot be changed
- A UTXO must be spent entirely; if you spend less, rest must be diverted back to you as a new UTXO

Where is this UTXO stored?

- Stored by every full node in the form of a database

How is this created and maintained?

- Scan all TXs from Genesis block to the current block to extract the UTXO db.

Node check inputs in a submitted TX against the UTXO db to validate.

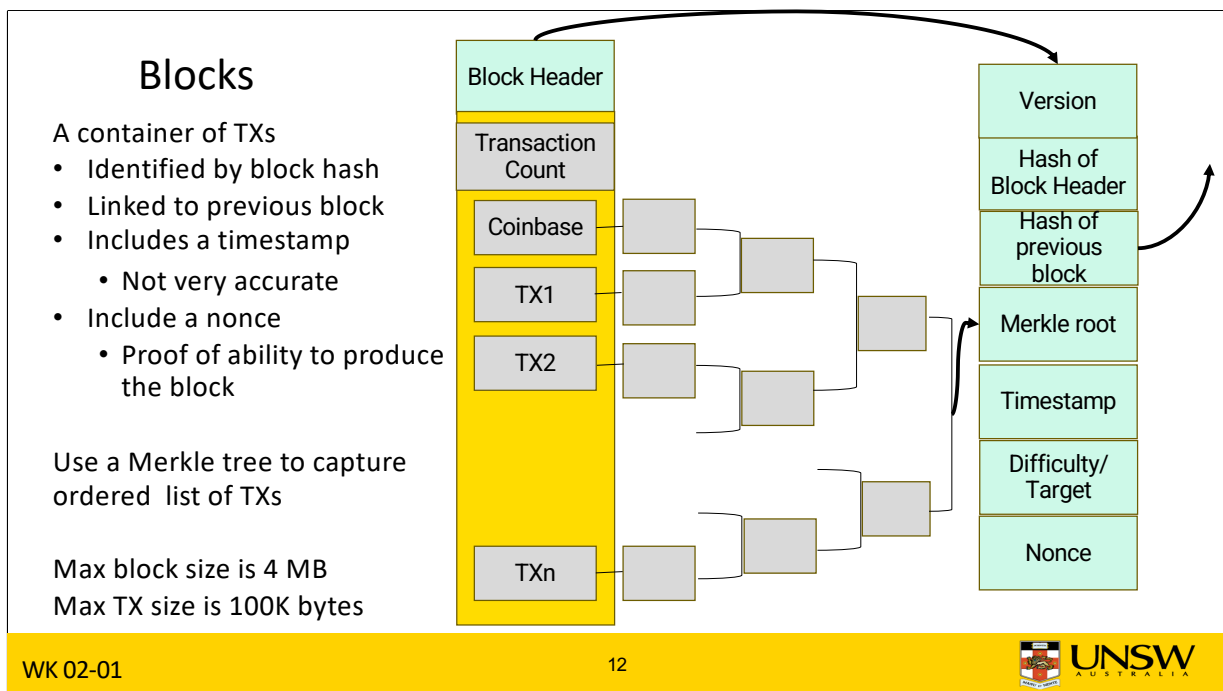
- Remove used UTXOs for confirmed blocks and add new outputs in the db

WK 02-01

11



- A UTXO can be spent only once as an input to a TX.
- UTXOs are bound to the recipient's public key.
  - Actually, a UTXO is bound to the hash of the public key known as the address (an address also has a version number and a checksum to detect errors when specifying an address).
  - So if you want to find how many Bitcoins you own, you need to sum all UTXOs associated with your address. Following the notes and coins analogy in your wallet, if you want to know how much money you have, you need to count them.
- Whereas the global state of the blockchain includes all the UTXOs.
- Bit“coin” is misleading, as fractional ownership, e.g., 1.64 BTC, is the norm.
- Each full node store and maintain the UTXO database
- This database is used to validate new TXs – check for double spending and calculate available balances



As discussed earlier in Week 1, a block is a container of TXs.

A block is identified by block hash, which is derived by hashing the block header.

In this figure, we have the details of a block header

Block links to the previous block using its hash.

Version number (4 bytes) helps a node to identify whether the rest of the network uses a new version of consensus rules that it may be not aware of. Bitcoin TXs also include a version no for the same purpose.

While the block includes a timestamp, it's not very accurate.

Due to clock drift, minors can set any timestamp within a certain window. Usually, this window can be as high as 2 hours into the future.

A block also includes a nonce used as proof of the ability to produce the block

Then the ordered list of TXs is summarised using a Merkle tree

The block header is 80 bytes in length.

The Max block size in Bitcoin is now 4 MB, early days it was 1 MB.

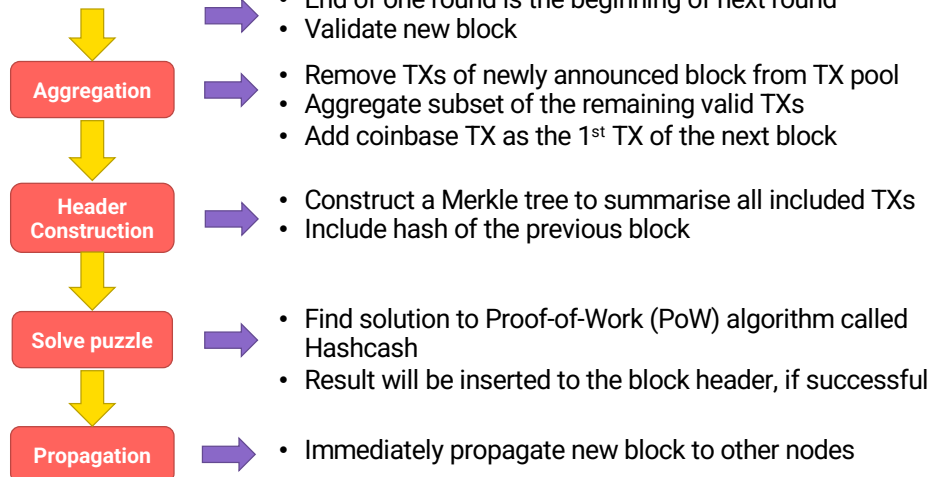
4 MB comes from 4 million weight units, which is a measurement used to compare the size of different TXs to each other in proportion to the consensus-enforced maximum block size limit. It won't be typically reached unless all TXs are formatted to minimise what's included in the block. The typical max is 2.3MB if all TXs are segwit TXs. See [https://en.bitcoin.it/wiki/Weight\\_units](https://en.bitcoin.it/wiki/Weight_units)

There's quite a lot of disagreement about whether the block size should be changed. Based on different block sizes, there are even sister BCs that were formed out of Bitcoin. E.g., supporters of large blocks who were dissatisfied with the activation of SegWit forked the software on 1 August 2017 to create Bitcoin Cash, becoming one of many forks of Bitcoin such as Bitcoin Gold.



## Mining – Creating a New Block

### Receive a new block



WK 02-01

13



Mining is the process of creating a new block. This diagram shows the overall mining process.

Once a new valid block is built and propagated to other nodes in the network, miners start building the next block while using the new block's hash as the previous hash.

**Aggregation:** First, they remove TXs of the newly announced block from a pool of pending TXs (aka transaction pool) as those are already in the block that was just announced.

Aggregate a subset of the remaining valid TXs.

Then add Coinbase TX as the 1st TX to the TX list for the next block. Coinbase TX is a special TX used to claim the block generation reward. This is how new Bitcoins get generated.

Coinbase TX has a special condition that it cannot be spent (used as an input) for at least 100 blocks. This is related to the forking discussed in the next slide.

**Header construction:** Miner then calculates the Merkle Root.

Next, build the header by including the hash of the previous block and Merkle Root to summarize all the included TXs.

**Solve puzzle:** Find a solution to the Proof of Work (PoW) algorithm. If successful result (in this case nonce) is inserted to the block header. We'll discuss these later

**Propagation:** Finally, a successful miner immediately propagates a new block to other nodes via the Internet

## Right to Build a Block

Miners compete to create new blocks by solving a hash puzzle known as Proof-of-Work (PoW)

- Hashcash in Bitcoin & Ethash in Ethereum

Answer is difficult to compute, but easy to validate

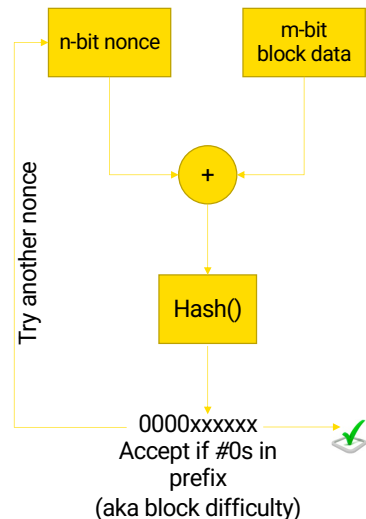
No shortcuts, must try all possible answers

Difficulty of the problem is automatically adjusted with time (Bitcoin Halving)

- Overcomes increasing computing power to maintain average inter-block time

Miners use specialised hardware to try multiple nonces at once

- ASICs in Bitcoin & GPUs in Ethereum
- See <https://youtu.be/x9J0NdV0u9k>



WK 02-01

14



Let's discuss a bit about the mining process in Bitcoin. Refer to the demonstration of the mining process shown in the class in Week 1. Once a set of pending TXs is included in a block, miners need to solve a puzzle to build a block. Once they solve the puzzle only, they can announce/propagate/broadcast the new block into the network. The miner who solves this puzzle 1st is considered to have the right to build the block. This puzzle is called Proof of Work (PoW) as it is difficult (computationally expensive, costly, and time-consuming) to solve the puzzle but easy for others to verify the answer. This is you solve a problem by spending an hour, while your teacher can verify your answer in seconds. Bitcoin miners solve a puzzle called hash cash to build a valid block. Ethereum 1.0 used an algorithm called Ethash (Ethereum 2.0 doesn't use PoW). PoW algorithms like hashcash are "embracingly parallel problems" with no shortcuts to find the answer.

The diagram outlines the basic idea: On one side you have the data that reflect the content of the block header like Merkle root, previous hash, and timestamp. The bitcoin block header is 80 bytes including a nonce, i.e.,  $m = 80 - 4 = 76$  bytes. On the other side, you have the nonce, a large random number. In Bitcoin nonce is 32-bits (i.e.,  $n = 4$  bytes) and the resulting hash value is 256 bits. You concatenate the 2 and calculate the hash value. You then check whether the resulting hash value satisfies a certain properly. E.g., For here we check whether the hash has 4 zeros as the prefix. Such a hash is called a valid hash, and the associate block is called a valid block. In practice, the acceptance threshold can be more specific like a very large number. The acceptance threshold is adjusted over time to ensure the average inter-block time remains 10 minutes with increasing computing power. If not, you must retry again while changing the nonce. You can't change the m-bit data, as it reflects the content of the block. If you want to do that you have to do more work than just trying another nonce. There's no easy way to guess what nonce would work. So, you must try nonce values until you get lucky. Hence, quite a massive number of hashes need to be tried before finding a hash value that satisfies the acceptance threshold. Thus, the process is computationally expensive and consumes lots of power. While it's difficult to find what nonce work, easy to validate whether a given nonce satisfies the condition. Just concatenate the claimed nonce with the block header, calculate the hash, and check whether the hash value satisfies the acceptance threshold.

The difficulty of the problem is automatically adjusted with time to overcome increasing computing power to maintain average inter-block time. A simple adjustment would be to increase/decrease the number of leading zeros in the hash. But a finer control can be achieved by adjusting the massive number that is hash value is expected to satisfy. Bitcoin – Every 2,016 blocks (approximately 14 days given roughly 10 minutes per block), nodes deterministically adjust the difficulty target/threshold. Ethereum – Difficulty is adjusted every block. This essentially means more computation the BC network put into solving the puzzle, and the difficulty of the puzzle gets increased, increasing the time to solve the puzzle. That way we can retain an average ~10-min inter-block time target.

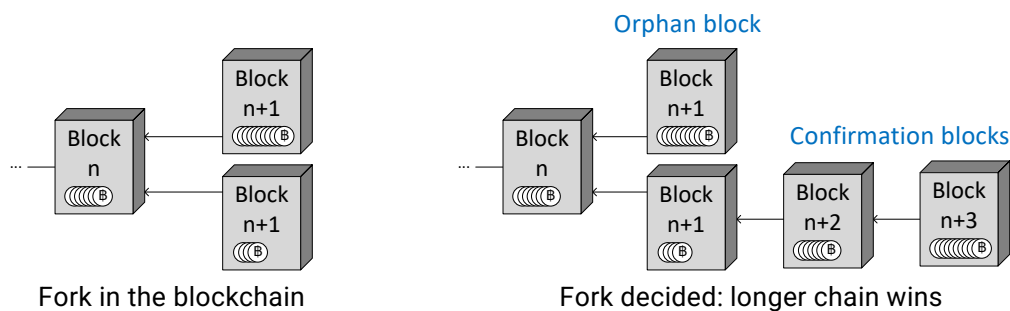
Miners use specialised parallel hardware to speed up the process by computing multiple nonce values parallelly: ASIC – Application-specific integrated circuit, GPU – Graphics processing units. The actual mechanism may slightly change from the given figure, e.g., instead of changing nonce on the block, miners may add a random value to coinbase TX to keep changing the block header until block difficulty is achieved.

## Right to Build a Block – Cont.

Multiple miners might find & announce next blocks at the same time

Tie breaker – Treat the longest history of blocks as the main chain

- One that received most computation
- Referred to as Nakamoto Consensus



WK 02-01

15



By finding a valid hash miners gain the right to build the next block. However, multiple miners can solve the problem simultaneously. This is quite possible in a global network like Bitcoin where it could take a few seconds for a newly generated block to propagate to a large fraction of the network. Such blocks will not be identical in content at least because the coinbase TX is bound to the miner's address.

When that happens, we need a tiebreaker. Nakamoto proposed to treat the longest history of blocks and claimed it as the main chain. The longest chain is also the one that received the most computation (more blocks behind it means more computation spent to solve the PoW puzzle).

In the figure, block  $n$  has 2 successors. Miners still allow subsequent blocks to be built while using these successors as the previous block. Eventually, one of the forked chains will be longer than the others. In this example, the bottom chain will be accepted as the longer chain.

The top  $n+1$  block is dropped from the chain of blocks, and all its TXs go back to the TX pool to be included in a block in the future. Such a block is called an *orphan block*. Therefore, creating a block doesn't mean the block is finalized. Even the block reward is not guaranteed.

This approach of deciding what blocks are considered as finalised is called *Nakamoto Consensus*. Occasional forking is common. However, extended forking is rare, and may be due to an explicit attack. When forking is occasional, Nakamoto (see Bitcoin whitepaper) showed that the probability that a block is no longer in the longest chain rapidly reduces to zero as more blocks are built along the same chain of blocks.

Therefore, in Bitcoin and several other BCs, after a TX gets included in a block, we wait for more blocks to be formed along the same chain. Each such block built after the block containing a TX of interest is called a *confirmation block*.

As multiple blocks may have the same block height (i.e., no of blocks in the longest chain since the Genesis block), it's not unique. Hence, block hash is the unique identifier of a block.

## Nakamoto Consensus

To determine with high probability that a TX is permanently included:

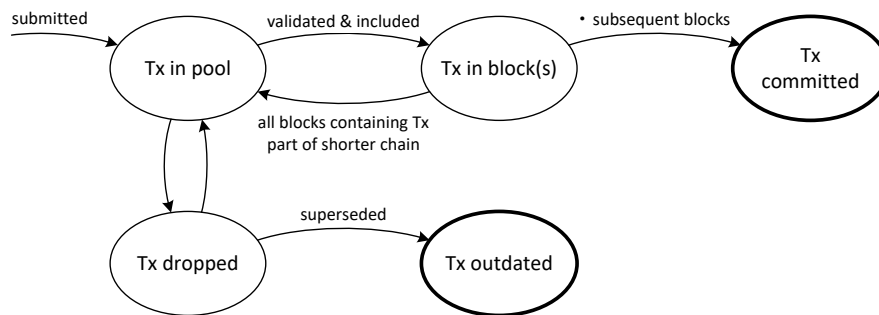
- Wait for several blocks (6 blocks by default) to be added after 1st inclusion of the TX in a block
- Each of these subsequent blocks is called a “confirmation block”
- Once sufficiently many confirmations occurred after the TX inclusion in the block then TX is considered committed/finalised

Unlike many traditional TX commit semantics:

- Commit only has a probabilistic guarantee
- A longer chain could still appear – although it may be very, very unlikely

- Let's talk a bit about the Nakamoto Consensus.
- To determine with a high probability that a block and its TXs are permanently included (i.e., in the longest chain), we need to wait for several new blocks to be added after the 1st inclusion of the TX in a block.
- This way, we can give enough time for any forks to get resolved. Again, here we are not talking about physical time, but the need to wait for enough new blocks.
- In Bitcoin, we assume that it is enough to wait for 6 blocks once a TX is included in a block. 6-blocks are approximately 1 hour (10 min x 10).
  - This number is determined based on a probabilistic analysis in the Bitcoin white paper.
- Each of these subsequent blocks is called a “confirmation block”. Once sufficiently many confirmations occur after the TX is included in a block, then the TX is considered committed (aka finalised).
- However, compared to traditional database-like TX commit semantics:
  - Commit only has a probabilistic guarantee as there is a non-zero probability (though extremely small) that a block may not be in the longest chain event after 10 or 12 blocks.
  - A longer chain could appear – although it may be very, very unlikely
  - In general, most BCs don't fully comply with ACID (Atomicity, Consistency, Isolation, Durable) properties associated with centralised databases.
- In practice, TXs need to be fast, persistent, & low-cost to be useful.

## Lifecycle of a Transaction



WK 02-01

17



This is a state diagram of the lifecycle of a TX.

As soon as a TX is submitted to the BC, it is validated by the receiving node.

If valid, TX enters the pool of pending TXs (aka TX pool or mempool). If not, TX is dropped.

When a valid TX is included in a block, it moves to the “TX in block(s)” state.

If the desired number of new blocks are built after this, the TX is committed. While Bitcoin assumes a TX is finalised after 6 new blocks, in Ethereum 1.0 we wait for about 12 blocks (approximately 3 min).

However, if the BC forks and the block with the TX isn’t included in the longest (or heaviest) chain, the block is discarded and all the TXs in the block are sent back to the TX pool.

While waiting in the pool, a TX may get dropped too. This could happen when the pending list of TXs is too long or the TX waited in the pool for a long time without being included in a block. These parameters are BC platform-specific and under the discretion of the miner. Typically, miners prefer to include TXs willing to pay a high TX fee. Hence, when the TX pool is too full, miners 1st drop TXs with a low TX fee. Also, miners may even define a minimum TX fee to be included in the TX pool.

A dropped TXs may be resubmitted with the same or a higher TX fee.

While a TX is pending in the pool, another TX can be submitted with a higher TX fee and (same nonce in Ethereum) to replace the existing TX. In that case, the original TX becomes outdated. However, this may not work always as altruistic miners may retain a TX in the pool regardless of its TX fee.

In practice, a few other complex scenarios could determine the lifecycle of a TX. However, this figure is abstract enough for the content of this class.

## Mining Reward

### Bitcoin Block 899,769

Mined on June 04, 2025 09:20:36 • All Blocks

Unknown

Coinbase Message • ,@h/Foundry USA Pool #dropgold/z>mm%vL } r>L( 3pOh/ uc~ \_nnZ T N bv

A total of 3,988.80 BTC (\$421,642,656) were sent in the block with the average transaction being 1.6524 BTC (\$174,669). Unknown earned a total reward of 3.13 BTC \$330,861. The reward consisted of a base reward of 3.13 BTC \$330,861 with an additional 0.0333 BTC (\$3,520.03) reward paid as fees of the 2,414 transactions which were included in the block.

#### Details

|               |                  |               |                        |
|---------------|------------------|---------------|------------------------|
| Hash          | 00000-d50c1      | Depth         | 1                      |
| Capacity      | 155.91%          | Size          | 1,634,848              |
| Distance      | 6m 51s           | Version       | 0x237a0000             |
| BTC           | 3,988.8027       | Merkle Root   | 26-e3                  |
| Value         | \$421,642,656    | Difficulty    | 126,982,285,146,989.30 |
| Value Today   | \$421,054,188    | Nonce         | 4,124,156,319          |
| Average Value | 1.6523623628 BTC | Bits          | 386,021,236            |
| Median Value  | 0.00574739 BTC   | Weight        | 3,993,826 WU           |
| Input Value   | 3,988.84 BTC     | Mined         | 3.15828863 BTC         |
| Output Value  | 3,991.96 BTC     | Mined on      | 04 Jun 2025, 21:20:36  |
| Transactions  | 2,414            | Height        | 899,769                |
| Witness Tx's  | 2,167            | Confirmations | 1                      |
| Inputs        | 9,014            | Fee Range     | 0-420 sat/vByte        |
| Outputs       | 6,357            | Average Fee   | 0.00001379             |
| Fees          | 0.03328863 BTC   | Median Fee    | 0.00000432             |
| Fees Kb       | 0.0000204 BTC    | Miner         | Unknown                |
| Fees kWU      | 0.0000083 BTC    |               |                        |

| Txn Hash               | Method              | Block    | Age       | From                   | To                        | Value                      | Txn Fee    |
|------------------------|---------------------|----------|-----------|------------------------|---------------------------|----------------------------|------------|
| 0x37a2c5e583f9842a...  | ocean0x47           | 14959244 | 1 min ago | 0x31af5ba3a442c0b5...  | 0x70526c7a6d320a44...     | 0.00000000014958 Ether     | 0.00718718 |
| 0x355c7e8b3552b10b...  | ocean0x47           | 14959244 | 1 min ago | 0x6d511a11231425eb...  | 0x70526c7a6d320a44...     | 0.00000000014958 Ether     | 0.0084022  |
| 0x09831111c393a0a4...  | Transfer            | 14959244 | 1 min ago | 0xc08275c68696d4a2...  | Tether: USDT Stablecoin   | 0 Ether                    | 0.00714071 |
| 0x674559d8b43109a53... | Transfer            | 14959244 | 1 min ago | 0x1a6359a6f743170a5... | Tether: USDT Stablecoin   | 0 Ether                    | 0.00740188 |
| 0x6810a381a692158...   | Transfer            | 14959244 | 1 min ago | 0x6538231170d8474...   | 0x60c743266a59144c...     | 0.001300000000000000 Ether | 0.00891472 |
| 0x055944518330c0a8...  | ocean0x47           | 14959244 | 1 min ago | 0x10a209666a7a7e76...  | 0x118c71d0ba222036...     | 0 Ether                    | 0.00712796 |
| 0x2f4f03a7b291381...   | ocean0x47           | 14959244 | 1 min ago | 0x10a209666a7a7e76...  | 0x118c71d0ba222036...     | 0 Ether                    | 0.00829412 |
| 0x1894a55a09005a...    | Contract All        | 14959244 | 1 min ago | Contract ETH           | 0x10a209666a7a7e76...     | 0 Ether                    | 0.00830302 |
| 0x330c2a0259185c...    | Register With Co... | 14959244 | 1 min ago | 0x6c75504840e0c02d...  | ENS: ETH Registrar Con... | 0.004439133376703 Ether    | 0.00891477 |
| 0x63a6d83a5623a07...   | ocean0x47           | 14959244 | 1 min ago | Contract ETH           | 0x10a209666a7a7e76...     | 0.1485 Ether               | 0.0088854  |

Transactions: 287 transactions and 111 contract internal transactions in this block

Mined by: 0x52bc44d5378309ee2abf1539b71de1b7d7be3b5 (Nanopool) in 12 secs

Block Reward: 2.124845907382975987 Ether (2 + 1.142902728055625435 - 1.018056820672649448)

WK 02-01

18



- Miners who build valid blocks get rewarded for their effort in 2 ways:
  - Block reward – New cryptocurrencies generated as part of block generation are assigned to the miner (included through coinbase TX).
  - TX fees – Fees paid by users to get their TXs included in the block, e.g., the difference between inputs and outputs in a Bitcoin TX.
- On the left, we have a screenshot of a mining reward from a Bitcoin block. It also shows the block difficulty
  - This shows that Miner received 3.125 BTC as the block reward and 0.0333 BTC as TX fee for 2414 transactions included in the block.
- On the right we have such information from Ethereum:
  - At the top, we have a list of TXs that were included in the block.
  - Miner is Nanopool
  - While there are both a block reward and TXs fees, a bit of Ether is destroyed as well.
    - The London upgrade included Ethereum Improvement Proposal ("EIP") 1559, a mechanism for reducing transaction fee volatility. The mechanism causes a portion of the Ether paid in transaction fees for each block to be destroyed rather than given to the miner, reducing the inflation rate of Ether and potentially resulting in periods of deflation.

## Mining Reward – Cont.

Miners who build valid blocks get rewarded for their effort in 2 ways

1. Block reward
  - With each block, new cryptocurrency is generated & assigned to the miner
  - Bitcoin – Block reward is added as a special TX into the block, called “coinbase TX”
    - 3.125 BTC since May 2020, 6.25 BTC in 2020, 12.5 BTC reward in 2016, 25 BTC in 2012, 50 BTC initially
    - Reward halved every 210,000 blocks
  - In Ethereum, block reward is credited to miner’s address
    - 2 ETH since block# 7,280,000, 3 ETH between 4,370,000 & 7,279,999, 5 ETH initially
2. TX fees
  - Miners can collect fees from TXs they include in the block
  - Higher TX fee → Higher chance of TX getting included in a block

Miners who build valid blocks get rewarded for their effort in 2 ways:

### Block reward

With each block built, new cryptocurrencies are generated and assigned to the miner. In Bitcoin, the block reward is added as a special TX into the block, called coinbase TX. Reward halved every 210,000 blocks, e.g., 6.25 BTC since May 2020, 12.5 BTC reward in 2016, and 50 BTC initially. Eventually, the reward will round down to zero, and the limit of 21 million bitcoins will be reached approximately in 2140. The miner’s effort will then be rewarded by TX fees only. Ethereum (we are jumping the Ethereum discussion a bit here just for the sake of a comparison). The block reward is credited to the miner’s address. Ethereum reward adjustment is not regular like in Bitcoin, e.g., 2 ETH since block# 7,280,000, 3 ETH between 4,370,000 & 7,279,999, and 5 ETH initially. Also, Ether has no supply cap.

### TX fees

Miners can collect fees from TXs they include in the block. Hence, they are incentivised to prioritise TXs that pay higher fees. Typically, higher TX fee increases the chance of a TX getting included in a block soon. TX fees can be complex and how they are charged changes across BC platforms, e.g., The London upgrade in Ethereum since block# 12,965,000 burn a portion of the Ether paid in transaction fees for each block to be destroyed rather than given to the miner. Essentially base fee (which is dynamic) is removed and miners are only allowed to retain what was paid above the base fee. This reduces the inflation rate of Ether and potentially results in periods of deflation. See <https://eips.ethereum.org/EIPS/eip-1559>

## Block Difficulty

A number that represents how difficult it is for miners to add new blocks to the Blockchain

- Current Difficulty: 121658450774825 = 121.66 Trillion
- Max\_Target (Genesis Block) =  
0x00000000FFFF0000000000000000000000000000000000000000000000000000000
- Current\_Target = max\_target / current difficulty
- Hash must start with around 19-20 leading Hex zeros
  - 76-80 leading zero bits
- Genesis block in Bitcoin has the requirement for 10 leading zeros
- Currently 19 leading zeros
- Difficulty gets updated/adjusted after every 2016 Blocks (two weeks approx.)

At this interval, each node takes the *expected time* for the last 2,016 blocks to be mined (2016 x 10 minutes), and divides it by the *actual time* it took: (20160 / actual)

If miners were able to solve each block more quickly than expected; say 9 minutes per block for example, you'd get a number like this:

$$20160 / 18144 = 1.11$$

Each node then uses this number to adjust the difficulty for the next 2,016 blocks: new difficulty = difficulty x 1.1.

Genesis block has requirement of 8 leading hex zeros but the actual hash came out to be of 10 leading zeros.

Max\_Target is still 8 Hex leading zeros (32 leading zeros bits)



## Block Difficulty – Cont.

No of leading Zeros

- Dictate the difficulty of finding a Hash
- Nonce is 32 bit
  - Try all  $2^{32}$  combinations at worst
- The more the number of leading zeros, the more restrictive the domain for the hash values, harder to compute

## Blockchain

899768

899768

Go

Next 0 blocks ↑

## Memory Pool

0.75 VMS

Refresh

Estimated time to get mined based on feerate:

WK 02-01

22



- Snapshot on 4<sup>th</sup> June 2025.
- Confirm 19 leading zeros for the Hash of the block
- Block fees vs estimated time to get mined
- Mempool status
- Size of the Bitcoin blockchain (block height 899768).

## Question

Which of the following statement(s) is True?

- A. As the number of ledger copies increases, it becomes difficult to maintain the consistency of the ledger.
- B. Given a hash value, we can derive the corresponding message.
- C. An attacker can fabricate a TX using Alice's public key to show that Alice is engaged in illicit TXs (e.g., money laundering).
- D. As soon as a Bitcoin TX is included in a block, it is safe to assume the TX is final (e.g., Charlie can ship a bicycle to Alice).

- A. As discussed in 1<sup>st</sup> class, keeping many ledger copies synchronised is very hard
- B. A good hash function is irreversible
- C. Digital signatures use the private key, which must be protected
- D. A TX that got included in a block may not be in the longest/heaviest chain of blocks for a while

## Fun Facts

Pizza Transaction in May 2010

- 10,000 BTC used to buy two Pizzas worth about USD \$40
- Now worth 164.32 Millions AUD.

QR code in a college sports event – 2013

[https://www.reddit.com/r/Bitcoin/comments/1rkjc9/espn\\_college\\_gameday\\_sign/](https://www.reddit.com/r/Bitcoin/comments/1rkjc9/espn_college_gameday_sign/)

- "HI MOM SEND BITCOIN" along with their bitcoin address in the form of a QR code. They managed to get their sign on TV by standing in the crowd at a college sports event. Also posted on Reddit.
- Over the next few days, they received 34.11382488 BTC across 120 transactions, which was worth \$42,219 at the time.

- What is Block number for the famous “Pizza Transaction”?
- How many inputs were used to pay 10000 BTC in the Pizza Tx?

# Agenda

## Popular Blockchain Platforms

- Bitcoin
- **Ethereum**
- Hyperledger

- Time to have a look at Ethereum. we'll revisit TXs, blocks, and ledgers structures in more details.

# Ethereum



2nd generation blockchain that can execute programs called Smart Contracts

- Ledger that can store/transact any kind of data

Native currency is Ether (ETH)

|                     | Ethereum 1.0 (Eth1) | Ethereum 2.0 (Eth2)  |
|---------------------|---------------------|----------------------|
| Consensus           | Proof of Work (PoW) | Proof of Stake (PoS) |
| Inter-block time    | Average 13-15 sec   | Regular 12 sec       |
| Confirmation blocks | 12                  | Max 64               |
| Ledger              | Replicated          | Replicated           |
| Performance         | Low                 | High*                |
| Power consumption   | High                | Low                  |

WK 02-01

26



As Ethereum fully supports smart contracts (SCs), it's considered a 2nd generation BC. SCs are small programs that execute on a DLT. More on SCs in the next class. Even Bitcoin supports a limited set of SC features. However, it's Ethereum that demonstrated the true power of SCs. With the introduction of SCs, the Ethereum ledger enabled the storage and transaction of any kind of data.

Ethereum was conceived in 2013 by Vitalik Buterin. In 2014, development work began and was crowdfunded through ICO (Initial Coin Offering), and the network went live on 30 July 2015. Development was funded by an online public crowd sale (ICO) from July to Aug 2014, in which participants bought the Ethereum value token (Ether) with another digital currency, Bitcoin. The native currency is Ether (ETH). ETH balance is denominated in Wei ( $10^{18}$  Wei = 1 Ether).

Today, we have 2<sup>nd</sup> version of Ethereum that has a few notable differences from its initial version:

**Consensus:** On Sep 15, 2022, Ethereum switched its consensus algorithm from PoW to PoS (Proof of Stake). PoS protocols are a class of consensus algorithms that work by selecting validators/miners in proportion to their quantity of holdings (i.e., stake) in the associated cryptocurrency (ETH). This essentially means that rather than selecting a miner to build the next block based on its computing power, a miner (minors are now called validators) is selected based on the amount of ETH in stakes. The actual protocol is more complicated, and we'll discuss PoS in a later class. The original PoW algorithm was called Ethash. The key difference between Ethash and Bitcoin's Hashcash is that it's also memory intensive making it difficult to solve the puzzle with hardware optimisations such as ASICs (Application-Specific Integrated Circuits). GPUs (Graphics Processing Units) were used in Ethereum. Initial inter-block time was 13-15 sec. Now blocks are generally more regular at every 12 seconds. Typically, 12 blocks of confirmation were used in Ethereum 1.0 before assuming a TX is confirmed. However, with the introduction of PoS, it can be now as high as 64 blocks. However, it is expected to reduce as the protocol undergoes further enhancements. Ethereum 2.0 is still undergoing transition. E.g., while the current ledger is fully replicated, Ethereum plans to decentralise the ledger using a technique called sharding. Sharding will split the ledger into different segments and each segment will have a relatively smaller number of replicas. This enables parallel execution of TXs. Due to sharding and the slight reduction in inter-block size, Ethereum 2.0 is expected to handle a much higher number of TXs. Hence, throughput (one of the performance metrics that count the number of TXs processed within a unit of time) is expected to be 2 orders of magnitudes higher than current performance. As PoS doesn't waste power to solve a puzzle like in PoW, the power consumption of Ethereum 2.0 is several orders of magnitude lower than what it used to be before Sep 2022. It is estimated that the power consumption is dropped by ~99.95%.

- See <https://ethereum.org/en/roadmap/merge/> for high-level details of Ethereum's transition from version 1.0 to 2.0

## Accounts and TXs

Uses account-balance model  
An account is bound to owner's public key

A TX is uniquely identified by its hash  
TXs are sequenced using a nonce

Once included in a block, block no, actual  
gas used, actual fee, etc., are available

|                                                                    |
|--------------------------------------------------------------------|
| <b>Tx Hash:</b> ①                                                  |
| 0xc13d7905be5c989378a945487cd2a1193627ae606009e28e296d48ddaec66162 |
| <b>Block Height:</b> ②                                             |
| 4169059 (2 block confirmations)                                    |
| <b>TimeStamp:</b> ③                                                |
| 1 min ago (Aug-17-2017 12:09:35 PM +UTC)                           |
| <b>From:</b> ④                                                     |
| 0xf1f5a00f88542b835cd57298d496e9798c09671                          |
| <b>To:</b> ⑤                                                       |
| 0x40c0e50da2f2fda5103914bcc5aad114743152a9c                        |
| <b>Value:</b> ⑥                                                    |
| 4.90793699 Ether (\$1,516.75)                                      |
| <b>Gas Limit:</b>                                                  |
| 21001                                                              |
| <b>Gas Used By Txn:</b>                                            |
| 21000                                                              |
| <b>Gas Price:</b>                                                  |
| 0.000000003 Ether (3 Gwei)                                         |
| <b>Actual Tx Cost/Fee:</b> ⑦                                       |
| 0.000063 Ether (\$0.02)                                            |

- From – Source address (regular account)
- To – Destination address (regular account or contract address)
- Value – Ether (in unit "wei") to transfer to destination (can be 0)
- Nonce – TX sequence no for source account
- Gas price – price you are offering to pay (Ether per gas)
- Gas limit – Max amount of gas allowed for TX
- Data – Payload data (TX memo, binary code, or function invocation)
- Digital signature (field names: v, r, s)

WK 02-01

27



- While Bitcoin uses UTXO, Ethereum uses the typical account-balance model where the ledger maintains the balance of each account.
- An account is bound to the owner's public key.
- TXs and blocks are uniquely identified by their hash values.
- TXs from the same account are ordered by a sequence number called the *nonce*. This nonce doesn't have anything to do with the nonce in a block used by PoW protocols.
- As we saw with Etherscan a TX has a hash, from & to address, nonce, value or data, and a TX fee.
- In Ethereum, TX fee has 2 parts, a gas limit and a gas price. We'll soon discuss gas...
  - Gas limit - is the maximum amount of gas the TX issue is willing to spend.
  - Gas price - is the amount of Ether the TX owner is willing to pay for a unit of gas
- Once a TX is included in a block, block no, actual gas used, actual fee, etc., are available.

## Transaction Fees

Sender decides TX fee to offer

- Can be 0. Minimum on some platforms

Miners prefer TXs with higher fees

- Altruistic mining
- Higher TX fees → Fast inclusion in a block

TX fees are dynamic

- No of pending TXs
- Cryptocurrency price
- TX urgency of users
- Errors & attacks

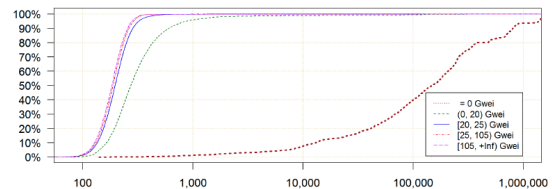
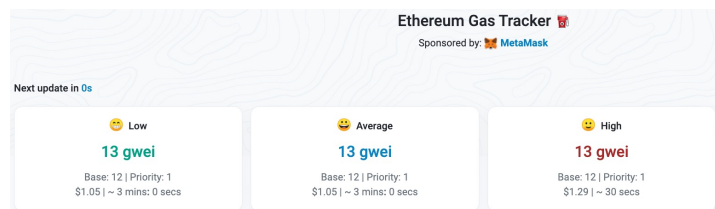


Figure 6: Commit delay (sec) for transactions based on gas price. Note logarithmic  $x$  axis.

Source: Ingo Weber et al., "On availability for blockchain-based systems", 2017 IEEE 36<sup>th</sup> Symposium on Reliable Distributed Systems (SRDS).



- In most public BC platforms, the sender decides the TX fee to offer.
  - While it can be 0, some platforms set a minimum TX fee, e.g., Ethereum has a minimum fee, but the miner may decide to change that.
- Miners prefer TXs with higher fees as it is part of their income.
  - There are a few altruistic miners who don't care about mining fees. Or miners charge no fee from these TXs.
  - Because of these, occasionally you would see even TXs with no or very low fees may get included in a block.
- Typically, a higher TX fee increases the chance of a TX getting included in a block soon.
  - E.g., the CDF shows that Ethereum TXs with high fees are included faster, while TXs with low fees can be significantly delayed. Even for the same TX fee, some TXs may experience substantial delays. 20% of Tx with 0 fee can take around 12.5 days.
- TX fees are dynamic and depend on many factors:
  - No of pending TXs – As the block size is limited (in terms of the block size or gas limit) when too many TXs are pending to be included in a block we have a high demand and limited supply scenario. Similar to other goods in a market, the TX fee goes up as the TX senders with high urgency will start offering higher prices.
  - Cryptocurrency price – Typically when the price increases concerning say USD, TXs fee goes down as miners' effort to maintain their computing infrastructure is reflected in USD. However, when the cryptocurrency price goes up, there can also be an increase in no of TXs increase the no of pending TXs in the pool.
  - TX urgency – Users' urgency depends on the use case, this is common during initial coin offerings (ICOs), gambling, and some NFT (non-fungible token) sales.
  - Errors are not uncommon where someone may accidentally put TX value as TX fee. Or an attacker may pay someone's currency as a TXs fee to any miner who's able to build that block.
  - Due to multiple such factors, TXs fees can vary widely.
- We could use services such as Etherscan Gas Tracker ([14 Gwei](#) | [Ethereum Gas Tracker](#) | [Etherscan](#)) or ETH Gas Station (no longer operational) to get an estimate on a suitable TX fee depending on how soon we want the TX to be included in a block
  - Wallets like Metamask and Ethereum node can also give you an estimate.
  - These services use various algorithms ranging from calculating mean/median value and moving averages to machine learning.



## Gas in Ethereum

A unit of accounting for calculating TX fees

- Based on computational complexity & storage needed to execute a certain operation/instruction
- A fee to limit resource usage

Gas limit

- A fixed gas cost per TX (base cost of 21,000 gas)
- Plus, a variable gas cost for data (dependent on size) & execution of a SC method (charged per bytecode instruction)
- Additional gas cost for deployment of new contracts

Gas price

- Reflects how much Ether-per-gas the TX sender is willing to pay
- Clients need set the gas price to market price & their urgency
- Use gas price recommendations from sources like <https://etherscan.io/gastracker>

Let's now discuss a bit about how TX fees are calculated.

TXs fees are defined in a unit called gas. Gas is a unit of account within Ethereum used in the calculation of the TX fee. This determines the amount of ETH a TX's sender must pay to the miner/validator who includes the TX in the BC. Gas is calculated based on the computational complexity and storage needed to execute a set of operations/instructions triggered by a TX. More complex computations or more storage means more cost. Hence, the gas acts as a fee to limit resource usage on the Ethereum network.

The offered TX fee consists of 2 parts, a gas limit and a gas price. This is like the fuel cost of going on a trip. Depending on the efficiency of your car, you will need a certain number of litres of fuel. Then there is the fuel price. So, the total fuel cost is = no. of litres of fuel x price per litre.

Gas limit:

There is a fixed gas cost (aka base cost) for a basic cryptocurrency TX and currently set to 21,000 units. If your TX includes data or executes an SC method, then you must pay additional gas in proportion to the size of the data or the complexity of bytecode instructions that are executed by the SC. Further, additional gas cost needs to be paid for the deployment of a new contract. This is also useful for the SC user, particularly to guard against errors such as infinite loops where bad code could quickly exhaust our Ether. However, if we set a low limit, we run the risk of exceeding the gas limit before our code finishes execution. If so, TX is reverted, but you lose the TX fee. Anyway, we don't want to set a very high limit as Ethereum nodes don't also like excessively gas-consuming TXs due to the risk of denial service (DoS) attacks. We can estimate how much gas an SC-related TX may consume. However, in practice, we set a slightly higher gas limit to accommodate any unforeseen changes in the size of data or code behaviour.

Gas price

Is the fee we are willing to pay for a single unit of gas? This should be specified in ET. Gas prices are typically denominated in Gwei, a subunit of ETH equal to  $10^{-9}$  ETH. While this is at the user's discretion, if you offer to pay a much lower value compared to the market price, your TX could get delayed or even get dropped. Alternatively, you could increase the chance of including TX faster by offering to pay a higher gas price. Gas price is dynamic and reflects the number of pending TXs. Some recommendations are available, e.g., from <https://etherscan.io/gastracker>. Set higher gas prices if the inclusion of TX is urgent. Set lower gas prices if TX inclusion is not (time) critical.

## Gas Limit

### TX gas limit

- Maximum gas TX sender is willing to pay
- Gas used  $\leq$  gas limit
- Else, TX will fail & state changes are reverted. Sender is charged up to the gas limit

### Block gas limit

- Sum of gas used by TXs included in a block can't exceed this limit
- Limits complexity for a new block
- Set by the miners
- Max block size is ~30 million gas
  - Max 1,428 TXs/block (Bitcoin 1,500-2000 TXs/block)
  - Most blocks under a few KB (Bitcoin 4 MB)
- An upper bound on TX throughput
  - Nontrivial to understand how the bound relates to TX throughput

- There are two gas limits in Ethereum.
- TX gas limit is the maximum gas the TX sender is willing to pay.
  - Gas used is the actual gas consumed by the TX and is set once the TX is included in a block.
  - Gas used  $\leq$  gas limit.
  - Else, TX will fail, and state changes will be reverted. However, the TX sender is charged up to the gas limit specified.
  - We can estimate how much gas a particular TX is likely to use. However, there's a possibility that other transactions may execute between the gas estimation and the inclusion of the said TX in a block. Due to the ledger changes caused by those TXs, the actual gas used could be higher than the initial estimate. Hence, it's a good practice to set a slightly higher gas limit than the estimate, e.g., a 20% increase.
- Block gas limit
  - The sum of gas used by all TXs included in a block can't exceed this limit, making it a limit of complexity for new blocks.
  - This limit is set by the miners. However, a miner can't set a very high value that the rest of the network is unlikely to accept.
  - As the limit is based on gas used, not gas price, it is not influenced by variations the user has power over, e.g., underbidding the market price.
  - The current max block size is around 30,000,000 gas. About 18 months ago it was 10M, which increased to 30M in a couple of steps.
  - In a 30M gas block, we can include up to 1,428 TXs in a block (min of 21,000 gas per TX). Whereas in Bitcoin we can include ~2000 TXs/block.
  - Because larger blocks mean more TXs per block, it increases the number of TXs processed within a unit time, aka TX throughput.
    - While a higher block gas limit helps to increase the TX throughput, it is nontrivial to understand how the bound relates to throughput (more on this in a later class).

## Block Format

Maintain state of all accounts

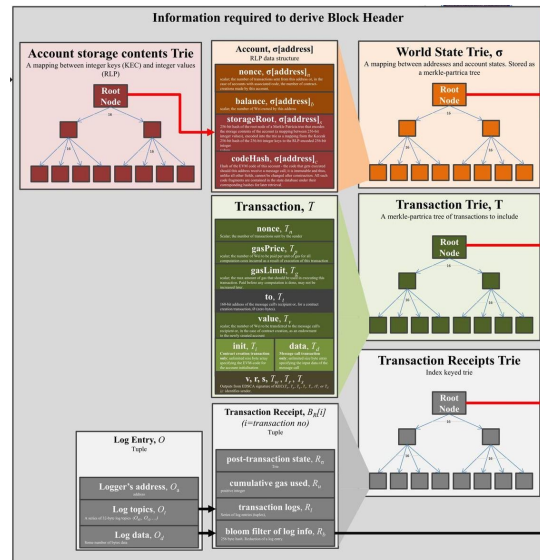
- Aka World/global state
- Include account balances, data stored, & smart contracts

List of TXs

List of TX receipts (i.e., effects of TXs)

Ethereum uses 3 Merkle trees (known as Trie), one each for integrity of:

- World State
- TXs
- TX Receipts



Source: <https://ethereum.stackexchange.com/Questions/268/ethereum-block-architecture>

- Let's talk about block format in Ethereum.
- A block keeps track of the entire state known as the world or global state. The world state consists of:
  - All account-balances
  - SC code
  - Data stored by SCs
- As we can see from the diagram, an Ethereum block is more complicated than Bitcoin because it also keeps track of TXs and outputs/results of TXs.
  - The results of a TX is called the TXs receipt and they reflect changes in the ledger state due to the TX.
- Each of these data is captured in a block as the root of the respective Merkle tree.
  - Merkel tree implementation in Ethereum is called Trie.

## Ethereum Protocol

Ethereum's inter-block time > block propagation time around the globe

- Bitcoin inter-block time >> block propagation time around the globe
- Small block size helps to propagate faster

Ethereum 1.0 with PoW

- Multiple competing blocks were more likely
- Uses heaviest chain consensus
- GHOST (Greedy Heaviest Observed Subtree) protocol linked orphaned blocks (called uncles) to increase the weight of chain
- Uncle block miners received 87.5% of a standard block reward

Ethereum 2.0 with PoS

- Only 1 validator is pseudo-randomly chosen to propose a block
- Temporary fork settled through Validators votes

- 12-15 sec inter-block time is only marginally higher than the time to propagate the newly generated block to a large fraction of the network.
  - Small blocks help Ethereum to achieve short inter-block time, as they can be propagated to other BC nodes much faster.
  - Whereas Bitcoin has a 10 min inter-block time giving ample time for a block to propagate.
- In Ethereum 1.0, due to the lower inter-block time, it was quite likely that multiple competing valid blocks were created simultaneously while another miner's newly created block continued to propagate the network.
  - To overcome this problem Ethereum 1.0 used a protocol called GHOST (Greedy Heaviest Observed Subtree).
  - GHOST is a bit complicated. The main idea was to link up orphaned blocks (in Ethereum's terminology they are called *uncle blocks*) with successor blocks to make a computationally heavier chain. Following consensus heaviest chain eventually wins (not the longest chain of blocks).
    - **Uncle** - a child of a parent of a parent of a block that is not the parent
  - The weight is determined based on the number of uncle blocks attached to a block.
  - Miners reference uncle blocks to add weight to their chain. This recognition is backed by a strong financial incentive mechanism, e.g., miners of uncle blocks receive 87.5% of a standard block reward. For every included uncle, the miner gains an additional 3.125%
- As per the Ethereum 2.0 protocol, only 1 miner is randomly selected to propose a block in a given round. Therefore, it is not susceptible to simultaneous block generation unless some malicious attack occurs. Due to network latency fork may still happen that are eventually resolved through heaviest chain.

## Ethereum 2.0

Client – Implementation of Ethereum that verifies data against protocol rules

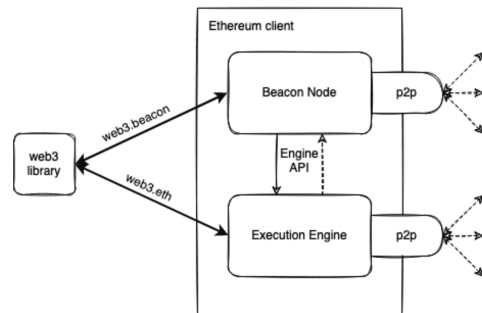
Node – Instance of Ethereum client software that is connected to others

Ethereum 1.0

- Single client for TX execution & consensus

Ethereum 2.0

- Execution client (aka Execution Engine, EL client, or Eth1 client) – Execute TXs & hold world state
- Consensus client (aka Beacon Node, CL client, or Eth2 client) – Implements PoS consensus



Source: [ethereum.org](https://ethereum.org)

- A “client” is an implementation of Ethereum that verifies data against the protocol rules and keeps the network secure.
- A “node” is any instance of Ethereum client software that is connected to other computers
- Ethereum 1.0 used a single client software to execute TXs, maintain the ledger state, and achieve consensus.
- Whereas Ethereum 2.0 separates TX execution from consensus by having 2 clients that work together on the same node (see figure from Ethereum documentation but note the abuse of terminology).
  - Execution client (aka Execution Engine, EL client, or formerly the Eth1 client) listens to new TXs broadcasted in the network, executes them, and holds the latest state, and world state.
  - The consensus client (aka Beacon Node, CL client or formerly the Eth2 client) implements the PoS consensus algorithm, which enables the network to achieve agreement based on validated data from the execution client.
  - This separation was to enhance modularity and let the existing execution clients run without breaking it or introducing new vulnerabilities.

## Question

Mark True or False for each the following statements about Bitcoin & Ethereum

|                                                                                      | True | False |
|--------------------------------------------------------------------------------------|------|-------|
| Immutability in Bitcoin is probabilistic                                             |      |       |
| Bitcoin uses account-balance model to keep track of assets while Ethereum uses UTXOs |      |       |
| Even through a TX got included in a block, it may eventually get dropped             |      |       |
| Compared to Bitcoin, throughput of Ethereum is higher                                |      |       |

WK 02-01

34



- Due to the Nakamoto consensus, a block containing a TX may not be in the longest chain after a while. And there is a non-zero probability that this may happen even after multiple confirmation blocks. Therefore, 1st statement is True.
- 2<sup>nd</sup> question is wrong as how states are managed across the 2 BCs are reversed, i.e., Ethereum uses an account-balance model while Bitcoin uses UTXO.
- 3<sup>rd</sup> question arise from 1<sup>st</sup> question. Due to non-zero probability a TX that got included in a block may eventually belong to an orphan block. The TXs on that block then go to the TX pool. While waiting in the TX pool, some of those TXs can get dropped, particularly if they have been waiting in the pool for a very long time. See Slide 24.
  - This is quite unlikely in Ethereum 2.0 due to its PoS design (as there will not be any uncle blocks). However, under a major attack on the blockchain, this may be possible, but quite unlikely.
- The last question is tricky. To answer this question, we can do a rough calculation.
  - In the context of BCs, throughput is the no of TXs included in a block per second.
  - So, Ethereum can include a maximum of 475 TXs in a block. Assuming a block is built every 15 sec, throughput is  $475/15 = 31.67$
  - Bitcoin on average includes 1500 TXs in a block. Assuming a block is built every 10 min, throughput is  $1500/10*60 = 2.5$
  - In practice, the average throughput of Bitcoin is considered to be 3-7 TX/sec while it's 15-25 per Ethereum.

# Agenda

## Popular Blockchain Platforms

- Bitcoin
- Ethereum
- Hyperledger

## Hyperledger

An umbrella project of a set of open-source blockchains & related tools

- Global collaboration, hosted by Linux Foundation since Dec. 2015

Hyperledger Fabric is an enterprise blockchain framework

- Private-permissioned blockchain
- Modular architecture, e.g., can change consensus algorithm
- Smart contracts are called “Chaincode” – Go, node.js, or Java

No concept of TX fee

- Can achieve much higher TX throughput with low latency
- 1,000+ public TXs or 500+ private TXs per second



**HYPERLEDGER**



**HYPERLEDGER  
FABRIC**



**HYPERLEDGER  
BESU**



**HYPERLEDGER  
INDY**



**HYPERLEDGER  
IROHA**



**HYPERLEDGER  
SAWTOOTH**

WK 01-03

36

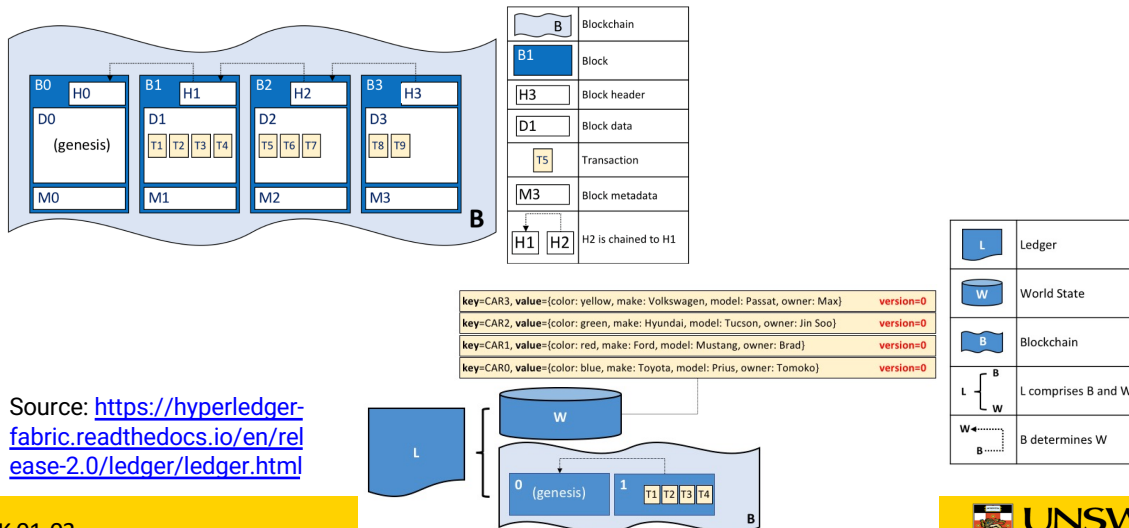


**UNSW  
AUSTRALIA**

- Hyperledger is an umbrella project of a set of open-source blockchains and related tools.
- The logs show some of the blockchain frameworks under Hyperledger.
- Hyperledger is hosted by the Linux Foundation with over 100 members.
- Hyperledger Fabric (HLF) is a business blockchain framework intended as a foundation for developing blockchain-based applications. We'll focus on HLF in this class. Besu is essentially Ethereum without gas fees.
  - It was initially developed by IBM.
  - It's a private-permissioned blockchain.
  - All Hyperledger frameworks are modular, e.g., In Fabric and Sawtooth, you can change the consensus algorithm.
  - SCs in Hyperledger are called Chaincode and can be developed in multiple languages
- No concept of the TX fee.



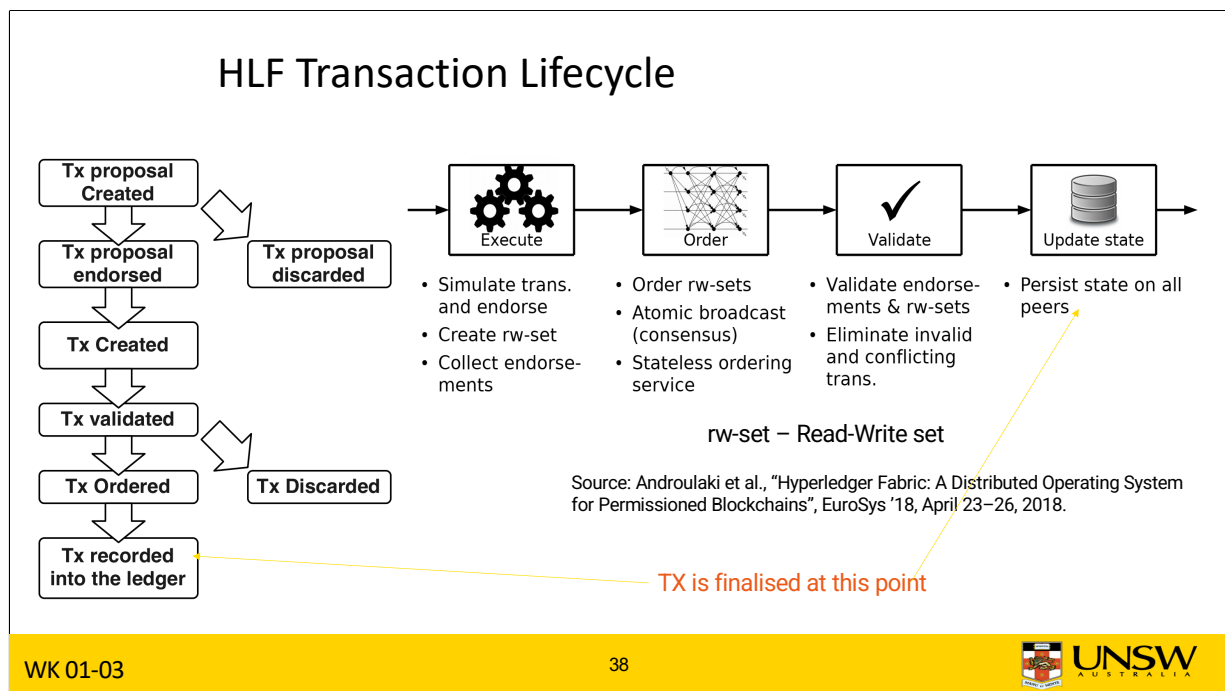
## Transactions and Blocks



WK 01-03



- The block format in HLF is similar to other BCs.
  - The only difference is there's no nonce as a special node called the orderer decides what TXs go into a block and their order.
  - Block metadata is not very different from other attributes included in block headers in Bitcoin and Ethereum.
- As the 2<sup>nd</sup> figure shows, the ledger includes both the world state and blocks including their TXs.
  - Like Ethereum, the HLF world state is also maintained as account balances.
  - Here *keys* are like account numbers and *values* are the state/balance. Being a 2<sup>nd</sup> generation BC, HLF can store any form of data as value.
  - If you are familiar with key-value stored in NoSQL databases HLF is not very different. HLF uses LevelDB or CouchDB as the underlying data store for the ledger. The key-value store in HLF is more visible compared to that in Ethereum which uses the same ideas though it's not directly visible to developers.



WK 01-03

38



These figures illustrate the lifecycle of a TX in Hyperledger. Once a TX is proposed to be included in the ledger, it needs to be first endorsed by a set of nodes. The nodes responsible for endorsement will check the authenticity of the TX, whether it is valid given the ledger state, and then put its digital signature as an endorsement of the TX. This is like getting your TX notarised. If the TX is invalid (e.g., Alice doesn't have enough money), it is discarded. An endorsed TX is sent back to the TX sender that proposed the TX.

Once the TX sender collects enough endorsement as per the set endorsement policy, TX is assumed to be in the "created" state. A simple endorsement policy could be "Organisation A's TXs must be endorsed by any 2 of Organisation B, C, and D" The TX sender combines multiple endorsements as per the endorsement policy and then sends all of them to a special node called the orderer.

Once a TX reaches the orderer, it validates the endorsement based on a pre-defined policy. If valid TX is ordered by putting it into a block. Else, it's dropped. Then the block is broadcasted to all the nodes in the network to update their ledger. Finally, the TX is recorded on the ledger. TX is finalized at this point. We don't need to wait for multiple confirmation blocks like in Bitcoin.

Here's another way of looking into this process.

The TX is executed and if valid it is endorsed. However, instead of updating the ledger, the state of the ledger before and after executing the TX is recorded as a read-write set (rw-set). The read set includes the ledger state before executing the TX and the write set includes the state after executing the TX. If the endorsement policy requires more than one endorsement, all endorsements need to be collected too.

The endorsed TX is sent to a special node called the orderer that orders TXs into a block. Order first validates the endorsements attached to the TX. Orderer doesn't execute the TX. It then builds a block by packing multiple valid TXs into it. This essentially forms a global order among concurrent TXs. E.g., if Alice, Bob, and Chaile's TXs were waiting to be included in a block, the order may order them as Bob, Charlie, and Alice. The orderer then broadcasts the block to all nodes in the network. Orderer is stateless and just orders TXs as it wishes.

Once a TX is received, nodes validate the endorsement and read-write set of a TX. If it's invalid or the ledger state is changed between the collection of the read-write set and validation, it's dropped. This could happen due to concurrent TXs, where the state read by one TX may have been updated by another TX that got finalised. If so, the client needs to resubmit the TX. In practice, rw-set conflicts are possible but rare, and you can design your application to minimise such scenarios.

Finally, the ledger is updated based on the information in the rw-set. TX doesn't get executed again, instead, the current ledger state is replaced with the write set. This executes → order → Validate model is different from the approach taken by BCs like Bitcoin and Ethereum where they follow Execute → Validate → Order TXs model. All these steps happen as part of the block-building process.

## HLF Network

### Membership Service Provider (MSP)

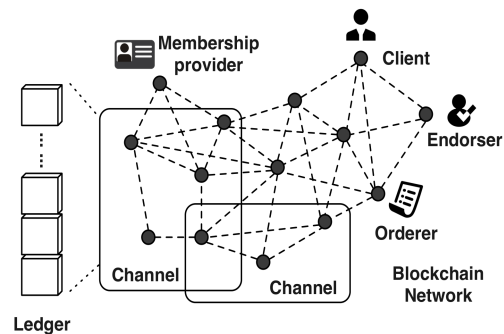
- Users & nodes must enrol with MSP & have known identities
- MSP is trusted
- Public keys as cryptographic certificates tied to organisations, network components, & users

### Channels

- Subnetworks – Allow a group of members to create a private ledger
- Built for scenarios where business confidentiality are important → Reduced transparency is acceptable

### Private Data Collections (PDC)

- Hide data in a TX from other channel members



As seen from the figure, Hyperledger Fabric (HLF) network has a set of components and nodes. The figure is taken from the textbook.

A membership Service Provider (MSP) is an identity provider service that links parties to their organisation. E.g., `alice.company.com` indicates that Alice is from `company.com`. All participants have known identities, and they need to enrol with the MSP. Public keys are used as cryptographic certificates (aka public key certificates) that are tied to organisations, network components, and end-users. MSPs link identifiers to respective organisations. MSP is trusted by all the participants – Here we see that the trust assumption is relaxed compared to public BCs. MSP is just a service, not a physical node.

**Channels:** In addition to enforcing participation at the network level, HLF can further group nodes into a set of channels. Channels are like subnetworks within the main network, that allow a group of members to create a private ledger. Data access control is applied on network and channel levels, so only members of a channel can access data in the respective ledger enhancing privacy. Therefore, channels are useful in scenarios where business confidentiality is important. This means only a subset of BC members will be in a channel. Hence, transparency is reduced, but acceptable in an enterprise setting.

**Private Data Collections (PDCs)** provide a mechanism to hide data within a TX for other channel members. E.g., the regulator may see the amount/volume of something sold but not its price. Note that PDCs are not depicted in the figure.

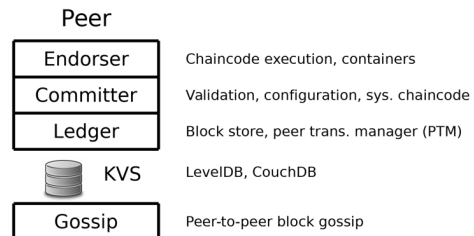
## HLF Node Types

### Client

- Connects to peers to communicate with blockchain on behalf of users
- Send TXs & observe updates
- Can connect to (multiple) channels, but is unaware of other existing channels

### Peer

- Receives ordered TXs from orderer, commits TXs, & maintains ledger state
- Can play a special role like endorser
  - TXs invoking chaincode needs to be endorsed before being committed
  - TXs must satisfy channel & chaincode-specify endorsement policies



### Orderer

- Validates & orders TXs into a block, then broadcasts it to the network
- Provides a communication channel between clients & peers
- Single orderer service prevents multiple competing blocks

The main node types are: Clients – Issue TXs. They need pre-authorisation from MSP. Peers – Maintain ledger state. Endorsers – Confirm TXs. Orderers – Order TXs. There is no good way to introduce these nodes types in a particular order as they are interconnected. A client connects to peers in the HLF network to communicate on behalf of an end user. It can be used to create & send TXs, as well as observe updates. A client may connect to multiple channels but is unaware of other existing channels that it's not part of.

Peers are the nodes of the BC network. What we see in this figure is the different layers of a peer. A peer maintains the ledger state and commits TXs ordered by the orderer. It can also play a special role like an *endorser*. In Hyperledger every TX invoking chaincode needs to be endorsed/confirmed/notarized before being committed. Therefore, before submitting a TX, it needs to be endorsed by one or more endorsers, who will run the chaincode and verify that the TX is valid. Chaincode can specify an endorsement policy that defines the conditions for valid TX endorsement. You can also define an endorsement policy for a channel. If both are defined, a TX must satisfy both of those policies before being included in the ledger. E.g., a policy like *org1.peer AND org2.peer* means a peer role from both org1 and Org2 needs to endorse a TX for it to be valid. Before endorsing a TX, the endorser will first validate the TX, execute it, and create a read-write set.

As the name implies, the Orderer orders TXs within the Hyperledger network creating a global order of TXs It validates (i.e., checks whether TX satisfies the endorsement policy, but doesn't execute it) and orders TXs into a sequence/block, then broadcasts them to the network. A set of orderers could be used to support multiple channels and achieve high availability. Even in that case, only 1 of them will be the primary orderer to build blocks. If that fails, another orderer will take over.

## Question

- Which of the following statement(s) is True?
  - A. As Hyperledger uses a Membership Service Provider (MSP), it's not required to sign TXs
  - B. Consensus algorithm in Hyperledger Fabric is based on PoW
  - C. Both Ethereum & Hyperledger Fabric maintain World State as a set of accounts & balances
  - D. Finality (i.e., time to confirm a TX) in Hyperledger Fabric is immediate

- A. MSP register users. But each TX still needs to be digitally signed
- B. While we didn't talk about a specific consensus algorithm for Hyperledger Fabric (HLF), in the early days it used to support a couple of them. But none of them were PoW-based. Today, it's mostly considered as support PoA (Proof of Authority). More on this in a later class
- C. Bitcoin uses UTXO, while Ethereum and HLF use an account-balance model.
- D. Finality is when you can consider a TX to be persistent (this is a finance or legal term, that refers to the moment at which TX/funds officially become the legal property of the receiving part). In HLF TX is final as soon as peer recorded them. But in Ethereum and bitcoin a TX included in a block may not be in the longest/heaviest chain for a while